FormuAI

DOE-Validated Formulation Surrogate Model Stack (CCD + OLS-RSM vs. RandomForest)

AUTHOR: Madhura Joshi (IIT Madras)

TARGET ROLE: Digital R&D; Data Scientist / Digital Transformation

DATE: July 2026

1. Executive Summary & System Conception

FMCG product development traditionally relies on physical trial-and-error laboratory formulations. FormuAI implements the complete digital R&D; workflow stack: it generates a Central Composite Design (CCD) matrix, logs results to a mock LIMS database, and fits dual modeling algorithms: classical Response Surface Methodology (OLS-RSM) and modern Random Forest Machine Learning. Visual analysis, ingredient sensitivities, and NLP agent queries are run from the Streamlit app.

2. System Architecture & Components

Component	Technology	Role in Digital R&D;
LIMS database	SQLite (Normalized)	Simulates laboratory notebook logs of formulation runs.
DOE Matrix	pyDOE2 (CCD)	Generates 20 balanced runs (factorial, axial, center reps).
Dual Solver	statsmodels & Sklearn	Fits quadratic polynomials and RF models side-by-side.
Interactive UI	Streamlit & Plotly	3D factor spaces, sensitivity line charts, and chat agents.

3. Classical Response Surface Methodology (RSM)

Response Surface Methodology is a statistical method used to fit a second-order quadratic polynomial to the experimental data. This polynomial models interaction terms and squared curvature effects:

- **Linear features:** SLES, CAPB, NaCl.
- **Interaction features:** SLES_CAPB, SLES_NaCl, CAPB_NaCl.
- **Quadratic features:** SLES_sq, CAPB_sq, NaCl_sq.

Quadratic Polynomial Fitting Snippet (src/models/dual_modeling.py)

```
df_poly = X.copy()
df_poly['SLES_CAPB'] = X['SLES_pct'] * X['CAPB_pct']
df_poly['SLES_sq'] = X['SLES_pct'] ** 2
df_poly = sm.add_constant(df_poly)
rsm_model = sm.OLS(y, df_poly).fit()
```

4. Machine Learning & Random Forest Comparison

For comparison, the system fits a RandomForestRegressor using 5-fold cross-validation. On small, smooth datasets like standard CCD designs ($N=20$), classical RSM OLS regression generally outperforms Random Forest in cross-validation R^2 metrics because trees cannot model smooth curvature with limited data. This comparison is a core differentiator for R&D; Data Scientists.

5. LIMS & ELN Data Integration

The mock LIMS system uses a flat table representing lab records: batch_id, run_number, operator, SLES_pct, CAPB_pct, NaCl_pct, water_pct, viscosity_sec, foam_height_initial_mm, foam_height_5min_mm, ph, clarity_score, replicate_flag.

6. AI Agent Prompt routing & LIMS querying

The NLP Lab Copilot parses researcher prompts using regex mapping to route actions:

- **LIMS query:** translating queries like *'Show trials by Madhura'* into SQL matching the operator name.
- **Predict properties:** running OLS-RSM models for user-input ingredient ratios.
- **Optimized design:** solving the formulation by search metrics given target viscosity and pH.

Agent Routing Code Structure (src/agent/lab_copilot.py)

```
if 'design' in p or 'optimize' in p:
    best_in, best_out = self.engine.optimize_formulation(target_visc, target_ph)
elif 'predict' in p or 'simulate' in p:
    outputs = self.engine.predict('RSM', inputs)
else:
    columns, results = run_custom_query(query, params)
```

7. Multi-Objective Cost-Performance Optimization

A central challenge in FMCG formulation is balancing performance (e.g. initial foam height) with the raw material unit costs of ingredients (SLES = 0.12 INR/g, CAPB = 0.15 INR/g, NaCl = 0.02 INR/g). This system implements a constrained cost optimization algorithm to minimize cost while satisfying physical specs.

Differential Evolution Solver

Using scipy.optimize.differential_evolution, the system searches the design space boundary for the lowest-cost formulation. Non-linear constraints (viscosity, pH, foam) are enforced using a penalty function:

```
def penalty_objective(factors):
    sles, capb, nacl = factors
    cost = (sles * 0.12) + (capb * 0.15) + (nacl * 0.02)
    preds = self.engine.predict('RSM', inputs)
    penalty = 0.0
    if preds['viscosity_sec'] < min_visc: penalty += (preds['viscosity_sec'] - min_visc)**2 * 50
    if preds['ph'] < min_ph or preds['ph'] > max_ph: penalty += (preds['ph'] - target_ph)**2 * 1000
    return cost + penalty
```

Pareto Front Analysis

By varying the target foam height constraint using the epsilon-constraint method, the solver generates a Pareto Front curve of Cost vs. Performance. This allows R&D; managers to select a formulation choice based on target tierings (Budget, Balanced, or Premium Performance), making technical trade-offs clear to business stakeholders.